

Архиватор на базе алгоритма Хаффмана

Евлоев Адам Саид-Магомедович

Кафедра: Системного программирования

Группа: 25.Б73-мм

Научный руководитель: Литвинов Юрий Викторович

Номер семестра практики: 2

1. Введение

Проект является учебной работой, посвящённой созданию консольного архиватора на основе алгоритма Хаффмана. В ходе работы были изучены сжатие без потерь, префиксное кодирование, построение бинарного дерева и побитовый ввод-вывод.

Работа выполнена на языке C. При реализации использовались структуры, указатели, динамическая память и обработка бинарных файлов. Проект показывает, как теоретический алгоритм сжатия применяется в программе и как данные можно представить компактнее без потери информации.

2. Постановка задачи

Цель данной работы — создать консольную программу, которая выполняет сжатие файлов и их последующее восстановление с использованием алгоритма Хаффмана.

Для достижения поставленной цели требуется решить следующие задачи.

1. Изучить алгоритм Хаффмана и существующие подходы к его реализации.
2. Спроектировать структуру программы и основные модули.

3. Реализовать сжатие и распаковку файлов.
4. Провести экспериментальную проверку на данных разных типов.
5. Сформулировать выводы об эффективности и ограничениях решения.

3. Обзор

Сжатие без потерь применяется тогда, когда после обработки необходимо полностью восстановить исходный файл. Это важно для текстов, исходного кода, исполняемых файлов и других данных, где потеря даже одного байта может привести к ошибке.

Алгоритм Хаффмана основан на частотах символов. Часто встречающиеся символы получают короткие двоичные коды, а редкие символы — более длинные. Для построения кодов используется бинарное дерево: переход влево соответствует биту 0, переход вправо — биту 1. Код символа получается как путь от корня дерева до листа.

В большинстве открытых реализаций используется общий подход: подсчёт частот, построение дерева, генерация таблицы кодов и побитовая запись результата. Для выбора двух узлов с минимальными частотами обычно применяется очередь с приоритетами. В данной работе выбран этот классический вариант, так как он прост и подходит для учебной реализации.

4. Описание предлагаемого решения

Программа написана на языке C. Сначала входной файл читается побайтно, и для каждого значения байта от 0 до 255 подсчитывается частота появления. Затем создаются узлы дерева Хаффмана, которые помещаются в очередь с приоритетами. На каждом шаге извлекаются два узла с минимальными частотами, объединяются в новый родительский узел и возвращаются обратно до получения корня дерева.

После построения дерева выполняется рекурсивный обход: при переходе влево к коду добавляется 0, при переходе вправо — 1. При сжатии

каждый байт исходного файла заменяется найденным кодом, а биты записываются в выходной файл через однобайтовый буфер. При распаковке программа читает сжатый файл побитово и восстанавливает символы, двигаясь по дереву Хаффмана.

Решение разделено на модули дерева Хаффмана, очереди с приоритетами, побитового ввода-вывода, сжатия и распаковки. Такое разделение делает код более понятным и упрощает проверку отдельных частей программы.

5. Эксперименты

Для проверки работы программы был создан экспериментальный модуль. Он формирует тестовые файлы, запускает сжатие и распаковку, сравнивает размеры файлов и измеряет время выполнения с помощью функции `clock()`. Для теста скорости каждая операция выполнялась по 10 раз.

Таблица 1: Результаты экспериментов по степени сжатия

Файл	Исходный, байт	Сжатый, байт	Степень
Текст 1 КБ	1 024	744	27.34%
Текст 10 КБ	10 240	6 239	39.07%
Текст 50 КБ	51 200	30 657	40.12%
Текст 100 КБ	102 400	61 180	40.25%
Текст 500 КБ	512 000	305 365	40.36%
Текст, разные символы	102 400	61 180	40.25%
JPEG изображение	2 097 152	2 098 436	-0.06%
MP3 аудио	8 945 229	8 934 297	0.12%
ZIP архив	10 952 853	10 954 137	-0.01%
Маленький текст	2 450	1 768	27.84%
Большой текст	3 359 405	1 960 875	41.63%

Таблица 2: Время выполнения операций

Операция / файл	Среднее время, сек	СКО, сек
Сжатие, файл 100 КБ	0.0035	0.0002
Распаковка, файл 100 КБ	0.0038	0.0004
Маленький текст, сжатие	0.0007	0.0005
Большой текст, сжатие	0.2141	0.0119

Из результатов видно, что на текстовых файлах от 50 КБ степень сжатия держится около 40%, а на большом реальном текстовом файле достигает 41.63%. Маленький текст сжимается слабее, так как при малом

размере сильнее влияют служебные данные. JPEG, MP3 и ZIP практически не сжимаются: результат близок к нулю, а JPEG и ZIP немного увеличиваются в размере.

Тест скорости показал, что сжатие файла 100 КБ занимает 0.0035 секунды, а распаковка — 0.0038 секунды. Поэтому в данном запуске распаковка не быстрее, а немного медленнее сжатия: отношение времени сжатия ко времени распаковки равно примерно 0.93. Отклонение для файла 100 КБ остаётся небольшим: 0.0002 секунды для сжатия и 0.0004 секунды для распаковки.

6. Заключение

В ходе выполнения работы были получены следующие результаты:

- изучен алгоритм Хаффмана и принцип префиксного кодирования;
- реализованы модули дерева, очереди с приоритетами, побитового ввода-вывода, сжатия и распаковки;
- проведены эксперименты на файлах разных типов и размеров, включая реальные текстовые файлы;
- подтверждено, что алгоритм эффективен для текстовых данных, но почти не сжимает JPEG, MP3 и ZIP.

Репозиторий с исходным кодом проекта:

<https://github.com/altrum06/huffman-arhive>